Proposed openpilot Software Architecture

Group H

Setayesh Chegini Setaywsh@my.yorku.ca
Spence Hashemi hash83mi@my.yorku.ca
Mehdi Jafarian mehdi04@my.yorku.ca
David Odidi david98@my.yorku.ca
Dan Logvinovsky logdn@my.yorku.ca

EECS 4314 Advanced Software Engineering, York University
https://eecs4314-group-h.github.io/openpilot-architecture-group/

October 25, 2026

Abstract

openpilot is an advanced driver assistance system from comma.ai. Its advertised features include Automated Lane Centering, Adaptive Cruise Control, Lane Change Assist, and Driver Monitoring. The driver remains responsible for watching the road and being ready to take over. For our report, Group H uses these features as a starting point to propose a conceptual software architecture. We are not claiming that our model describes the product's actual internal implementation.

Our proposed design separates the driving path into Sensors & Cameras, Perception, Planning, Controls, and Vehicle Interface. Other responsibilities, including Driver Monitoring, the Self-Drive Supervisor, User Interface, Localization & Calibration, and System Services, support and check this path. We combine a pipeline-like processing path, publish-subscribe communication through a Message Bus, and closed-loop feedback rather than forcing the whole system into one strict architecture style. This could make responsibilities easier to understand and modify, but communication delays, outdated information, and coordination between subsystems are important trade-offs. Our main finding at the conceptual level is that no single style fully covers the driving, supervision, and feedback needs. The rest of the report explains our proposed choices, interactions, and limitations.

1 Introduction

According to comma.ai, openpilot helps with steering and speed on compatible vehicles. Automated Lane Centering supports steering along the road, while Adaptive Cruise Control can adjust speed and respond to other vehicles. The website also advertises Lane Change Assist and camera-based Driver Monitoring. These features do not make the vehicle self-driving: the driver must stay alert and able to take control. comma.ai states that manual takeover must remain possible and that steering and braking actions are constrained by limits [1, 2]. We use these public product statements to define the problem, not to infer how the software is programmed.

The architecture question is how a system with these responsibilities could be organized. Reading observations, understanding the road, selecting a path or speed, and requesting vehicle actions

are related tasks, but each has a different purpose. Driver attention, warnings, and system supervision also need to be represented. Separating these concerns makes it easier to explain the design and consider what happens when one part changes or receives incorrect information. Our course describes architecture in terms of major components, connectors, and design decisions, while distinguishing a conceptual model from a concrete implementation [3, 4].

Group H proposes a modular design that mixes several architectural styles: a pipeline-like driving path, publish-subscribe communication through a Message Bus, and closed-loop control using vehicle feedback [4, 5]. Clearer subsystem boundaries may help maintenance, but timing and coordination remain important challenges. The report next explains the architecture and subsystem responsibilities, with an overall and a dependency diagram, then how the design was derived, the alternatives considered and the design patterns it uses. It then covers data and control flow, concurrency, external interfaces, use cases and interaction diagrams. The final sections discuss developer responsibilities, the data dictionary and naming conventions, design trade-offs, lessons learned, and conclusions.

2 Overview and Scope

In our proposed design, Sensors & Cameras supplies observations. Perception interprets them, Planning chooses a desired path or speed, and Controls prepares bounded steering, acceleration, or braking requests. Vehicle Interface represents communication with the Car and receives updated vehicle state. This is a useful pipeline-like explanation, but not a pure pipe-and-filter system: later decisions also depend on feedback and supervision [4].

The supporting subsystems have separate purposes. Driver Monitoring evaluates attention in our model. The Self-Drive Supervisor coordinates engagement, important alerts, and checks on information availability, while User Interface communicates status to the Driver. Localization & Calibration provides motion and alignment context, and System Services handles supporting concerns such as device health, logging, and updates. A Message Bus represents proposed communication channels between the relevant subsystems. Publish-subscribe could reduce direct dependencies, but it does not itself guarantee when a message arrives or which reaction happens first [4].

The Driver and Car remain outside the proposed software boundary. Driver takeover and bounded vehicle requests are important constraints from comma.ai's public description [1, 2]. Detailed subsystem analysis, interactions, and trade-offs appear in later sections. This report does not verify actual source files, runtime processes, internal messages, or measured safety and performance [6].

3 Proposed System Architecture and Components

EECS 4314 defines conceptual architecture as an abstract view of a large software system and its major interconnections [3]. Following that definition, the proposal separates sensing, interpretation, planning, control, vehicle adaptation, driver supervision, and support responsibilities, with the Driver and Car at the system boundary. It is our proposed model and does not describe openpilot's internal implementation.

3.1 Overall Proposed Architecture

Figure 1 shows the proposal. The main driving path moves from Sensors & Cameras to Perception, Planning, Controls, and Vehicle Interface before reaching the Car. Vehicle state returns through Vehicle Interface for the next decision. A second path observes driver attention, lets the Self-Drive Supervisor set operating constraints, and uses the User Interface to request action from the Driver.

Localization & Calibration adds position and alignment context, while System Services provides health and non-driving support. The Message Bus is the shared publish-subscribe connector.

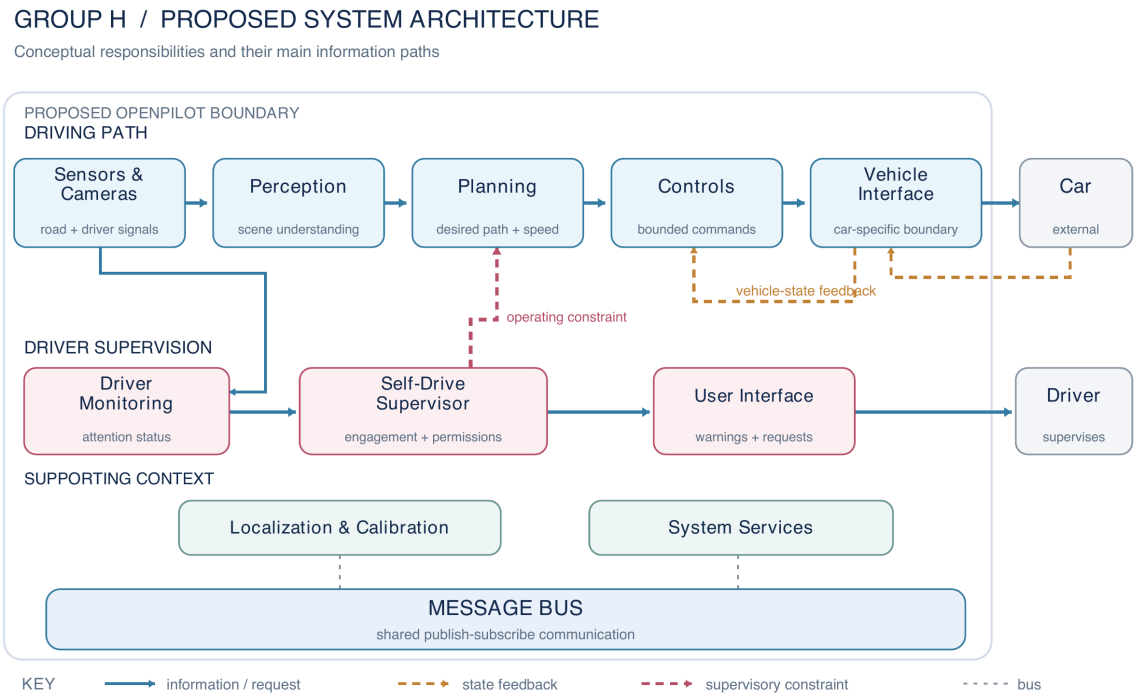


Figure 1: Overall proposed system architecture (Group H proposal). Arrows distinguish the driving path, vehicle-state feedback and supervisory constraints; dotted links indicate Message Bus access.

3.2 Major Subsystems and Responsibilities

The inventory below follows the names already used by Group H [7]. Each entry describes one proposed responsibility. The Driver and Car remain external actors, and the Message Bus remains a connector rather than a decision-making component.

These boundaries define the main interactions. Sensors & Cameras provides observations; Perception and Planning refine them into driving goals; Controls and Vehicle Interface form the path to the Car. Driver Monitoring, the Self-Drive Supervisor, and the User Interface form a separate supervision path. Information shared across those paths travels through the proposed Message Bus, while control authority remains with the subsystem that owns the decision.

3.3 Architecture Styles and Communication

Processing pipeline. The main driving path is pipeline-like: each stage transforms information before the next stage uses it. It is not a pure pipe-and-filter design because feedback, localization, supervision, and alerts cross the main path. The separation still gives each stage a clear input and output.

Publish-subscribe. The Message Bus applies the course idea that publishers and subscribers exchange information through a shared channel [4]. This reduces direct coupling, but the design must still address message freshness, ordering, failure handling, and diagnosis.

Closed-loop feedback. Planning supplies a desired state, Controls produces a bounded request, the Car changes state, and new measurements return for the next comparison. The driver-supervision path can constrain the loop but does not bypass Vehicle Interface.

Table 1: Proposed subsystems and their responsibilities (Group H proposal).

Proposed element	**Responsibility in Group H's model**
Sensors & Cameras	Collects road and driver images plus motion and location observations. It publishes observations without deciding the driving response.
Perception	Turns observations into a useful description of the road, lanes, and relevant traffic conditions.
Localization & Calibration	Maintains proposed position and sensor-alignment context so observations can be interpreted consistently.
Planning	Chooses a desired path and speed response from perceived conditions, current vehicle state, and supervisory constraints.
Controls	Compares requested motion with measured vehicle state and produces bounded steering, acceleration, or braking requests.
Vehicle Interface	Separates car-independent responsibilities from the compatible-vehicle boundary, returns vehicle state, and forwards permitted requests.
Driver Monitoring	Turns driver-facing observations into attention status. It reports that status but does not command the car directly.
Self-Drive Supervisor	Owns engagement, operating permission, takeover conditions, and health-based constraints in the proposal.
User Interface	Shows status and warnings, accepts relevant driver requests, and communicates when the driver must act.
System Services	Provides configuration, health information, diagnostics, logging, and update support outside the immediate driving-decision path.
Message Bus	Acts as the publish-subscribe connector for conceptual messages. It transports information but does not make domain decisions.

3.4 Component Dependencies

Figure 2 uses dependency arrows, not runtime message-flow arrows. An arrow from A to B means that A needs information or a service owned by B. Planning therefore depends on Perception and Localization & Calibration, while Controls depends on Planning, vehicle state through Vehicle Interface, and operating permission from the Self-Drive Supervisor. Dotted arrows show the separate communication dependency on the Message Bus.

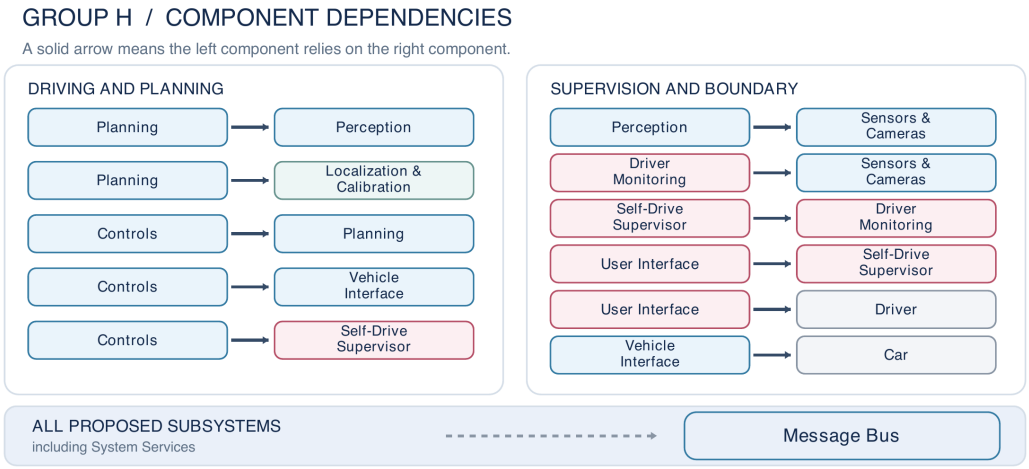


Figure 2: Component dependency diagram (Group H proposal). A solid A-to-B arrow means A relies on B; the dotted arrow shows their shared communication dependency on the Message Bus.

3.5 Assumptions and Limits

The boundaries, dependencies, message categories, and arrow directions are Group H design assumptions. The proposal keeps the documented constraints that the driver remains attentive and able to take control, and that vehicle actions are limited [1, 2]. It does not claim specific timing, algorithms, deployment units, internal data formats, or vehicle protocols.

4 Derivation Process, Alternatives and Design Patterns

4.1 How the Architecture Was Derived

Hassan and Holt derive a conceptual architecture by first *proposing* one from domain knowledge and documentation, and only later refining it against the implementation [8]. This report is that first step, taken in five stages.

1. **Collect product facts.** From comma.ai's overview, FAQ and documentation [1, 2] we listed the advertised features (lane centering, adaptive cruise control, lane change assist, driver monitoring) and the constraints every design must respect: the driver can always take over, actuation stays within limits, commands reach the car over its CAN network, and 350+ car models must be supported.
2. **Turn features into responsibilities.** Each feature needs sensing, interpretation, a decision, an action on the car, and a check that the driver is still supervising.
3. **Group responsibilities into subsystems.** Responsibilities that change for the same reason were grouped together, giving the eleven elements of Table 1; anything car-specific was isolated in one place.
4. **Choose styles for the connections.** For each candidate style we asked the course's questions: its components and connectors, its invariants, and its advantages and disadvantages [4]. We kept what fit the requirements of speed, many cars, driver override and easy evolution (Section 4.2).
5. **Check against the use cases.** We traced lane centering, adaptive cruise control and driver monitoring through the subsystems (Sections 5 and 7). This showed that the loop must close through vehicle state and that supervision needs its own path; after feedback on our presentation, we also redrew the ACC sequence diagram to show concurrent activity rather than a single sequence of steps.

4.2 Alternatives Considered

Table 2 lists the styles from the course that we considered for the whole system and what we decided about each.

4.3 Design Patterns

Design patterns sit one level below architecture, but their names give a precise shared vocabulary [9]. Five patterns from the course describe parts of our proposal.

- **Adapter** (Vehicle Interface). Adapter converts an existing interface into the one clients expect without changing the existing one [9]. Each car model's network is an adaptee that cannot be changed; Controls is the client of one car-independent target interface, and the Vehicle Interface adapts between them. This is what lets one design support 350+ models [1].
- **Template Method** (car ports). Every car follows the same skeleton: recognise the car at start-up, read its state, translate requests, and enforce its limits. Only the car-specific steps differ [9], which keeps new car ports from the community [1] consistent.

Table 2: Architectural alternatives considered (Group H proposal).

Alternative	Why it does or does not fit	Decision
Strict layers	Supervision, calibration, the User Interface and vehicle-state feedback all cross the driving path, and performance would force layers to be bypassed [4].	Rejected
Pure pipe and filter	Matches the staged driving path, but the vehicle-state feedback makes the topology a cycle, and the style suits interactive systems poorly [4].	Used for the driving path only
Repository (shared world model)	One consistent state is attractive, but the data model must be fixed in advance, evolving it is expensive, and a central store would serialize concurrent subsystems [4].	Rejected as the main style
Client-server (explicit calls)	A caller waits for each answer and must know its server, which adds latency and tight coupling on a time-critical path [4].	Rejected
Open-loop control	Cannot correct for how the car actually responds, or notice that the driver took over.	Rejected; closed loop used
Implicit invocation (publish-subscribe)	Loose coupling, easy to add consumers, producers never wait; ordering and freshness must be handled explicitly [4].	Chosen as the connector

- **Observer** (Message Bus). Observer defines a one-to-many dependency in which dependents are notified when a subject changes [9]. Each information category of Table 3 has one producer (the subject) and any number of consumers (the observers), for example Vehicle State reaching Planning, Controls and the Self-Drive Supervisor.
- **State** (Self-Drive Supervisor). The Supervisor's engagement state (not ready, dashcam-only, ready, engaged, disengaged) changes how other subsystems behave: Controls sends requests only while engaged. The attention escalation in Figure 4 is a second state machine.
- **Strategy** (longitudinal control). On some cars openpilot controls speed, while on others the stock cruise control does [2, I]. Treating these as interchangeable strategies chosen per car keeps the rest of the driving path unchanged.

We also considered **Master-Slave**, which gains fault tolerance by running replicated services and choosing among their results [9]. Redundant perception would suit a safety-critical system, but nothing in comma's product description suggests it, so we list it as a direction for future work rather than part of the proposal.

5 Proposed Data Flow, Control Flow and Concurrency

This section explains how information and control would move through the Group H architecture while openpilot is driving. It is a *Group H proposal*, derived from openpilot's advertised behaviour [1, 2] and the architectural styles of EECS 4314 [5, 4], not from its implementation. Information is named by what it *means*, not by any concrete message format.

5.1 Information Categories

Table 3 lists the information we propose the subsystems exchange. Each category has exactly one producer, which keeps ownership clear and lets any number of consumers subscribe to it through the Message Bus.

Table 3: Proposed information categories (Group H proposal).

Category	**Content**	**Producer**	**Consumers**
Observation	Road and cabin images; position and motion readings	Sensors & Cameras	Perception, Driver Monitoring, Localization & Calibration
Pose & Calibration	Device alignment; vehicle position and motion	Localization & Calibration	Perception, Planning
Scene Interpretation	Lanes, road edges, lead vehicles, predicted path	Perception	Planning
Motion Plan	Desired path and target speed	Planning	Controls
Control Request	Bounded steering, acceleration and braking request	Controls	Vehicle Interface
Vehicle State	Speed, steering, pedals, buttons, set speed, car radar where fitted	Vehicle Interface	Planning, Controls, Self-Drive Supervisor
Attention Status	Whether the driver is attentive	Driver Monitoring	Self-Drive Supervisor
Engagement State	Not ready, dashcam-only, ready, engaged or disengaged	Self-Drive Supervisor	Controls, User Interface
Alert	Message to the driver and its urgency	Self-Drive Supervisor	User Interface
Health & Log Data	Subsystem liveness, device status, recorded drives	All subsystems	System Services, Self-Drive Supervisor

5.2 Proposed Data Flow

The driving path is pipeline-like: an Observation becomes a Scene Interpretation in Perception, a Motion Plan in Planning, a Control Request in Controls, and a car-specific command that the Vehicle Interface places on the car's CAN network [1]. It is not a pure pipe and filter. That style requires filters that do not depend on what happens downstream [5], yet Planning and Controls need Vehicle State, produced at the *end* of the path; and it suits interactive systems poorly [4], while the driver may take over at any moment.

The better description is closed-loop process control, which the course illustrates with automobile cruise control [4]. In ACC, the *process* is the car's motion; the *set point* is the driver's cruise speed, lowered when needed to keep a safe gap; the *controlled variables* are speed and distance to the lead vehicle; the *manipulated variable* is the Control Request; the *sensors* are Vehicle State and the cameras; and Planning and Controls together form the *controller*. The loop closes through Vehicle State: each command changes the car's motion, and the measured result feeds later decisions.

Two side flows join the main path. Localization & Calibration supplies Pose & Calibration so image positions can be related to the real road. In parallel, Driver Monitoring turns cabin Observations into Attention Status, and the Self-Drive Supervisor turns that into Alerts for the User Interface. The monitoring path never commands the car; it influences driving only by changing the Engagement State.

5.3 Proposed Control Flow

We propose that control passes by implicit invocation [5]: no subsystem calls another; each acts on information it subscribes to. Each source publishes at its own pace: cameras produce frames, and the car reports its state continuously. Planning and Controls do not wait for any particular message; whenever they act, they use the newest information available. The Self-Drive Supervisor alone holds driving authority and governs this life cycle:

1. **Startup readiness.** The Supervisor reports *not ready* until every subsystem is publishing fresh data, a supported car is recognised and calibration is complete. An unsupported car falls back to a dashcam-only mode that keeps the stock features [2, C].
2. **Engagement.** When the driver asks to engage and no fault is active, the Supervisor publishes *engaged*; only then does Controls send non-neutral requests.
3. **Decision communication.** Planning decides *what* motion is wanted, Controls decides *how much* actuation is safe, within reasonable limits [2, G], and the Vehicle Interface decides *how to say it* to a specific car model.
4. **Feedback.** The effect of each command appears in later Vehicle State, closing the loop without anyone waiting for a reply.
5. **Disengagement.** The driver can always retake control with either pedal or the cancel button [2, G]; this wins immediately. The Supervisor also disengages when the driver is distracted [2, A], and we propose the same for a fault.
6. **Delayed information.** Every item carries its production time. A single late item is tolerated by reusing the latest value; persistent staleness makes the Supervisor alert the driver and hand control back. The system never acts on data it knows to be stale.

5.4 Concurrency

We propose that each subsystem is an independent unit of concurrency. As Hassan and Holt note, a conceptual architecture can name the unit of concurrency without fixing whether it is a process or a thread [8]. Camera-driven work (Sensors & Cameras, Perception, Driver Monitoring), the car's state reports (Vehicle Interface), driving decisions (Planning, Controls, Self-Drive Supervisor) and background work (calibration refinement, User Interface, logging and updates) all proceed at their own rates. The driving and monitoring paths run side by side and meet only at the Supervisor.

The Message Bus coordinates these units with publish-subscribe, so a producer never waits for its consumers [4], and a slow consumer reads the newest item rather than a backlog. Implicit invocation has known weaknesses: a publisher cannot rely on the order of responses or know when they finish [4]. In a vehicle these are safety concerns, so timestamps let consumers detect old or out-of-order data, and the Supervisor watches Health & Log Data. The performance-critical path is camera to Control Request: its delay is the sum of every stage and messaging hop, and Perception, which runs a neural network [1], is the heaviest stage. Logging and uploads must never block it.

5.5 Sequence Diagram: ACC Responding to a Slower Vehicle

Figure 3 traces ACC through the proposed architecture, assuming openpilot is engaged, a cruise speed is set, and the car is one where openpilot replaces the stock ACC [2, I]. Because the subsystems run concurrently, the diagram uses a par fragment whose three regions happen at the same time, each repeating at its own rate; messages are numbered per region so that no order between regions is implied, and every message is asynchronous.

In the first region, Sensors & Cameras publish a road Observation (1.1) and Perception publishes a Scene Interpretation of the slower lead vehicle (1.2). In the second, the car reports its speed, pedals and buttons (2.1) and the Vehicle Interface publishes Vehicle State to Planning, Controls and the Supervisor (2.2–2.4). In the third, Planning reads the newest inputs and chooses a lower target speed (3.1, 3.2); if engaged, Controls limits the deceleration (3.3) and publishes a Control Request (3.4), which the Vehicle Interface sends to the car (3.5). The car's response is not a reply: it appears in later measurements in the second region, which is how the loop closes. The break fragment interrupts all regions: the driver brakes (4.1, 4.2), the Supervisor sees the takeover (4.3), tells Controls it is *disengaged* (4.4) and alerts the driver (4.5, 4.6).

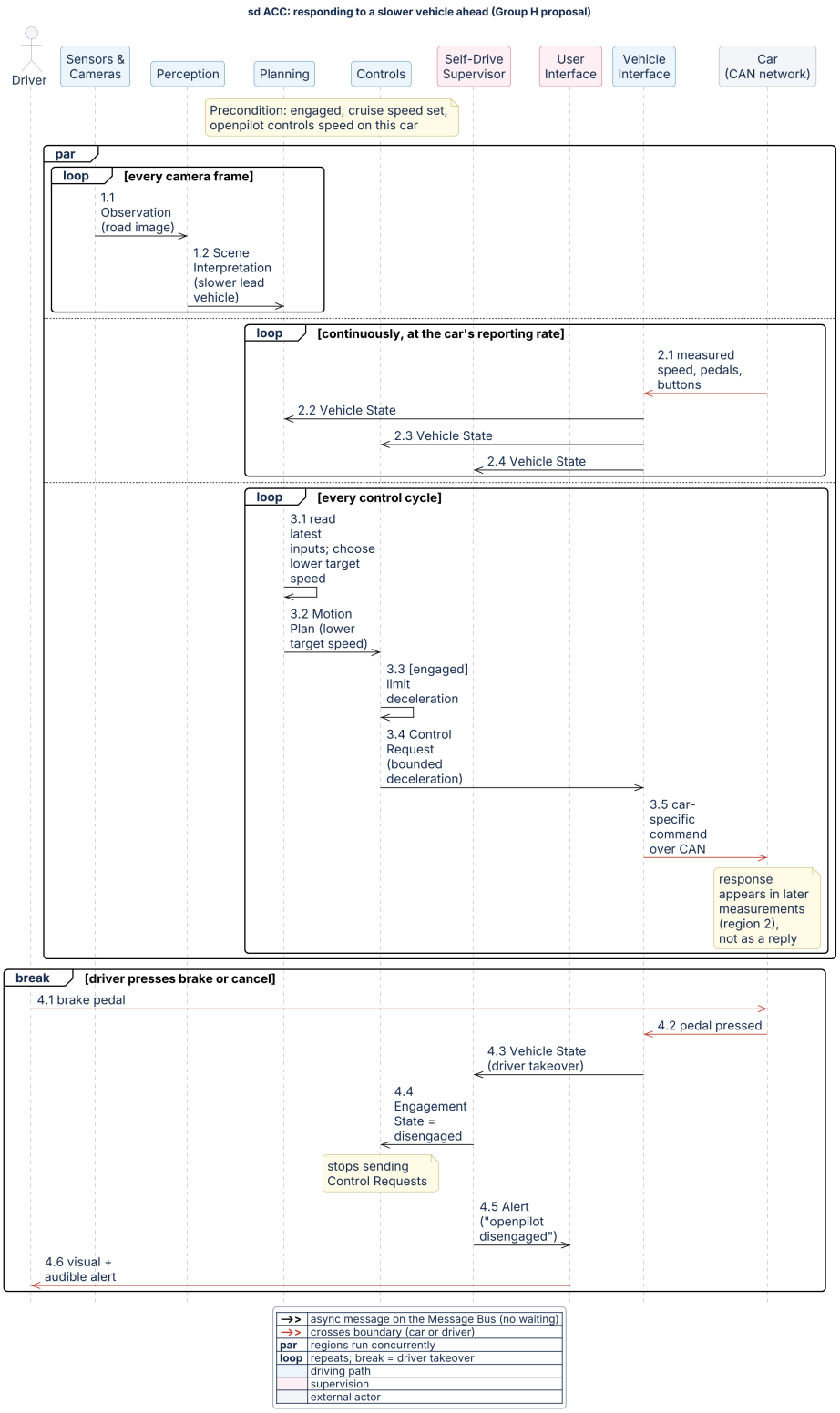


Figure 3: Sequence diagram for ACC responding to a slower vehicle ahead (Group H proposal). The par regions run concurrently; the break fragment shows a driver takeover. Open arrowheads are asynchronous messages over the Message Bus; red arrows cross the openpilot boundary to the car or driver.

5.6 Architectural Implications and Assumptions

Each stage only reads and publishes information categories, so it can be tested alone by replaying recorded inputs. A feature can be added as a new subscriber without touching producers, but a change to a category affects every consumer [4], so each category in Table 3 is a contract owned by its producer. Car differences stay inside the Vehicle Interface, so the flows above are the same for every car. We rely on the fact that the car's own safety features, such as automatic emergency braking, stay active [2, I], and assume that (A1) the car reports its state often enough to close the loop and (A2) a driver takeover is visible in Vehicle State. The concrete architecture study in the next assignment should check these first.

6 External Interfaces

openpilot has six external interfaces with four outside parties: the driver, the road scene, the car and comma's cloud (Table 4). It uses the car's existing driver-assistance interfaces to supply better steering, acceleration and braking than the stock system [10], over the car's CAN network [2, B]. Every command passes a separate safety check before it reaches the car [11].

The driver engages openpilot [12], keeps hold of the wheel, and can retake control at any time with the brake pedal or cancel button [13, 14]. The cabin camera feeds driver monitoring, which alerts distracted and asleep drivers while engaged [10, 15], and the road-facing cameras supply the road scene [15]. On the car side, each supported model needs its own adaptation, which reports the car's state and carries out control actions within limits [16]; openpilot recognises the car model automatically at start-up [2, C]. Toward the cloud, drives are recorded and updates arrive over the air [15] [2, E]. Two safety rules bind every interface: the driver can always retake control, and the path must never change too fast for the driver to react [13].

Table 4: External interfaces at the openpilot system boundary (→ = feeds).

Interface (who)	**What enters**	**What leaves**	**Handled by**
I1. Driver controls	Engage; brake and cancel; steering, gas; lane-change intent	Nothing directly (replies via I3)	Vehicle Interface; Self-Drive Supervisor
I2. Driver observed	Cabin-camera images of face and eyes	None	Sensors & Cameras → Driver Monitoring
I3. Alerts and displays	Touchscreen choices, e.g. installing an update	Attention alerts, warnings, takeover requests, notices	User Interface
I4. Road scene	Road-camera images; position and motion	None	Sensors & Cameras → Perception
I5. Car network (CAN)	Car state, car identity, radar data where fitted	Steering, gas and brake commands within limits	Vehicle Interface
I6. comma cloud	Software updates	Recorded drive data	System Services

I1 and I3 form the driver's two-way channel: the driver acts through the car's own controls and openpilot answers through the display and sound. I2 and I4 are input-only. I5 is the only interface that can change the car's motion, and it replaces stock lane keeping (and, on some cars, cruise control) while keeping the car's emergency braking [12] [2, I]. I6 lies outside the driving loop.

7 Use Cases

Each use case gives the actor, preconditions, trigger, main flow, alternative flows and the subsystems it activates. Use Case B is also traced by the sequence diagram in Figure 3.

Use Case A – Lane Centering

Actor: Driver (primary); car (secondary).

Preconditions: Supported car recognised at start-up [2, C,D]; device mounted correctly, road camera clear [14].

Trigger: Driver engages openpilot [12].

Main flow: (1) Road cameras record the road ahead (Sensors & Cameras). (2) openpilot computes steering that keeps the car centred [10] (Perception, Planning, Controls). (3) The request is turned into the car's own commands within limits [16] and checked before it is sent, so the path never changes too fast [13] (Vehicle Interface). (4) The car's state is read back for later decisions.

Alternative flows: (a) The driver corrects the steering, or brakes or cancels to disengage [13]. (b) Lane change: the driver checks the blind spot and signals intent [14]. (c) Sharp curves, poor visibility or a blocked camera: steering is limited and the driver must take over [14].

Subsystems: Sensors & Cameras, Perception, Planning, Controls, Vehicle Interface, Self-Drive Supervisor.

Use Case B – Adaptive Cruise Control

Actor: Driver (primary); lead vehicle (observed); car (secondary).

Preconditions: openpilot engaged on a car where it controls speed, otherwise stock ACC stays in charge [12]; within the car's speed range [2, F].

Trigger: A slower vehicle appears ahead, or the lead vehicle changes speed.

Main flow: (1) Road cameras record the road, and the car's radar is used where fitted [16]. (2) openpilot sets acceleration and braking for the vehicle ahead, never above the set speed [10, 14] (Perception, Planning, Controls). (3) Commands are sent within acceleration limits [16] (Vehicle Interface). (4) The car slows and its new state is read back. (5) When the road clears, openpilot returns to the set speed.

Alternative flows: (a) The driver brakes or cancels, which disengages [13]. (b) A sudden stop is needed: openpilot brakes only gently, so the driver must brake, with the car's own emergency braking as a backup [14, 12]. (c) Stop lights, signs, stopped cars and close cut-ins may not be handled [14].

Subsystems: Sensors & Cameras, Perception, Planning, Controls, Vehicle Interface, Self-Drive Supervisor, User Interface.

Use Case C – Driver Monitoring

Actor: Driver (primary, observed).

Preconditions: openpilot engaged [2, A]; cabin camera clear and face in view [14].

Trigger: The driver looks away or falls asleep [10].

Main flow: (1) The cabin camera records the driver (Sensors & Cameras). (2) Monitoring tracks awareness [2, A] (Driver Monitoring). (3) Distraction is detected and the driver is alerted [10, 13] (User Interface). (4) The driver looks back and openpilot carries on.

Alternative flows: (a) The driver stays distracted: openpilot disengages and the driver must take control [2, A]. (b) Brake or cancel returns control at any time [13]. (c) Darkness, glare or a

blocked camera can make monitoring wrong, so the driver must not rely on it [14].
Subsystems: Sensors & Cameras, Driver Monitoring, Self-Drive Supervisor, User Interface.

7.1 Driver-Monitoring State Diagram

Figure 4 shows the states openpilot passes through when an engaged driver becomes distracted. The driver starts in control and engages openpilot [12]; while engaged, the cabin camera feeds driver monitoring [10]. If the driver looks away, the driver is alerted [13]; if the driver stays distracted, openpilot disengages and the driver must take control [2, A]. From any engaged state the brake pedal or cancel button returns control at once [13].

Across the use cases, inputs enter through Sensors & Cameras and the Vehicle Interface, and outputs leave through the Vehicle Interface, User Interface and System Services. Use Cases A and B run the driving path from cameras to car, closed by car-state feedback; Use Case C runs the monitoring path, which acts on driving only by alerting or disengaging. The single route to the car, through the Vehicle Interface, is the safety boundary [13].

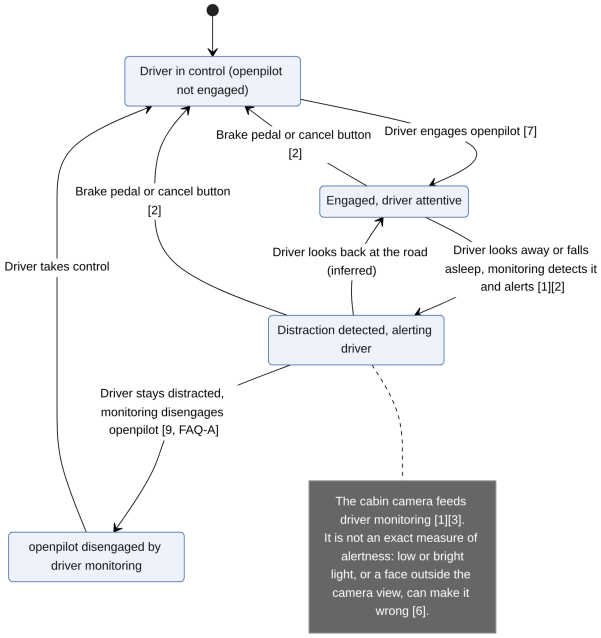


Figure 4: Driver-monitoring state diagram (Use Case C). Rounded boxes are states, arrows are transitions labelled with their trigger, the filled circle is the initial state, and the dark box is a note.

8 Design Trade-offs

We described our conceptual version of openpilot as a heterogeneous architecture: publish-subscribe implicit invocation between components, a pipeline-like path that turns sensor data into a driving decision, and a closed-loop process control style that feeds the vehicle's latest state, including the driver's decision to override, back into each decision.

The system must above all be fast enough to react to ever-changing road conditions, and our styles were chosen partly for speed. Storing large amounts of information does not belong on the real-time driving path, so recording is kept in the background (Section 6).

These choices have costs. A heterogeneous architecture gains flexibility over a strictly layered one but loses some of its efficiency and predictability. Publish-subscribe lets data reach whichever components need it and lets the system grow without retooling many components, but it is asynchronous and, on some level, time-agnostic: there is no guarantee that information will be available when needed. Some part of the system must watch messages for freshness and validity, which is the role of the Self-Drive Supervisor (Section 5.3). The pipeline keeps components separate, which helps testing and concurrency, but each stage adds latency, and because feedback interrupts it, timing and ordering need extra care. Finally, the feedback loop lets the system adapt to new information about the road and vehicle, but its inputs come from sensors and car components with their own tolerances, so latency and noise will likely have to be handled car by car.

9 Implications for Developer Responsibilities and Evolution

The proposed architecture suggests how work could be divided among developers. Each subsystem in Table 1 can have its own owning team, and because each information category in Table 3 has exactly one producer, the producing team also owns that category as a contract: it must coordinate with every consuming team before changing what the category contains, since a format change affects all subscribers [4]. Consumers, in turn, can be developed and tested independently by replaying recorded information.

Car support needs a different kind of team. Supporting 350+ car models [1] is mostly work inside the Vehicle Interface, and the open-source community contributes new car ports and bug fixes [1]. Because car differences stay behind one interface, these contributors do not need to understand Perception or Planning; they need to follow the common skeleton of a car port and respect its limits. Anything that changes what can reach the car, or how the driver retakes control, needs a stricter safety review than other changes, since those rules bind every interface [13].

The same structure explains how the system can evolve. New features can be added as new subscribers without changing producers; new cars are added behind the Vehicle Interface; improved perception can replace the Perception subsystem as long as its Scene Interpretation stays the same; and software updates reach users over the air [2, E], which is how frequent releases such as those listed on the product page [1] can be delivered.

10 Data Dictionary

Table 5 defines the key terms used in this report. The information categories exchanged between subsystems are defined in Table 3.

Table 5: Data dictionary.

Term	**Type**	**Definition**
Heterogeneous architecture	Architectural concept	An architecture that combines several styles [5]; here, one that is not strictly layered.
Implicit invocation	Architectural style	Components announce events instead of calling each other, which decouples them [4].
Publish-subscribe	Style variant	Implicit invocation in which subscribers register for the information they need and publishers broadcast it.
Pipe and filter	Architectural style	Filters transform streams and pass results through pipes; filters share no state.
Closed-loop process control	Architectural style	A controller adjusts a process to keep outputs near a set point, using measurements fed back from the process [4].
Message Bus	Connector	The publish-subscribe channel through which subsystems exchange information categories.
Information category	Data	A kind of information one subsystem publishes and others consume (Table 3).
Engaged / disengaged	System state	Whether openpilot is currently allowed to control the car.
Set point	Control variable	The value the controller aims for; in ACC, the driver's chosen speed.

11 Naming Conventions and Abbreviations

Subsystem names are written in title case exactly as listed in Section 3 and are used identically in every diagram and table. Information categories (Table 3) are also capitalized and named by meaning, not by any implementation message. Every figure is labelled "Group H proposal".

Abbreviations: **ACC** Adaptive Cruise Control; **ADAS** Advanced Driver Assistance System; **AEB** Automatic Emergency Braking, a factory safety feature that openpilot preserves; **ALC** Automated Lane Centering; **CAN** Controller Area Network, the in-car network that carries vehicle data and commands; **DM** Driver Monitoring; **ECU** Electronic Control Unit, one of the car's control computers; **FCW** Forward Collision Warning; **GPS** Global Positioning System; **LDW** Lane Departure Warning; **OTA** over-the-air (software updates); **UI** User Interface.

12 Lessons Learned

In working on this report, we learned several things about software design at a conceptual level. It is tempting to think of a system as fitting cleanly into one architectural style. Even before looking at any implementation, and just considering openpilot conceptually, we had to combine several styles in a way that amplifies their strengths and controls their weaknesses. Picking a style is not a single decision: it requires weighing a project's requirements, both the obvious ones and those that only appear after long deliberation.

We also learned that how a design is drawn matters as much as the design itself. A wrong diagram leads to confusion when reading a proposed system and can lead to incorrect implementation. In our presentation, a diagram described a feedback loop without an arrow returning to the start; in this report every loop is drawn closed and every diagram has a legend.

Finally, we learned to respect the boundary between conceptual and concrete architecture. Our presentation drew on source code, which belongs to the concrete architecture of the next assignment. A conceptual architecture should be justified by requirements and architectural reasoning, so in this report product facts come from comma's published product pages and documentation, never from the code, and design choices beyond those facts are labelled as Group H proposals.

13 Conclusions

Overall, we propose a specific model of openpilot: one where subsystems are not layered, where they exchange information through publish-subscribe, where sensor data moves along a pipeline-like path to reach a driving decision, and where a closed-loop process control system adjusts to the measured result and to unexpected inputs. Our system is very concerned with time, since reactions to input data must be fast. This raises concerns about timing and about ensuring the order of events between concurrent subsystems.

In the future, these implementation questions must be answered: whether the work can be done fast enough while keeping each data transformation simple, and how the system should control timing, messaging and driver takeover between our proposed components. Our next step, in the next assignment, is to compare this proposal with openpilot's concrete architecture and see how its contributors solved these problems.

14 Use of Generative AI

Each member of Group H wrote their own assigned sections of this report, and the group coordinated its work through a shared collaborative workspace.

Because we expected the report to read and look better in LaTeX, we used a generative AI assis-

tant to translate our written sections into LaTeX code with consistent formatting of headings, tables, figures, cross-references and references. Generative AI was also used to rearrange and integrate the sections into a single LaTeX file.

For the sequence diagram of adaptive cruise control (Figure 3), we described the diagram to a generative AI tool and asked it to produce PlantUML source code (a `.puml` file), which was rendered as an image. We later used generative AI again to re-render the diagram in a more compact form so that it fits the page better in the LaTeX document.

References

[1] comma.ai, "openpilot." https://www.comma.ai/openpilot. Accessed October 25, 2026.

[2] comma.ai, "comma support: openpilot FAQ." https://comma.ai/faq. Answers cited by letter: A "Do I have to pay attention?"; B "How does openpilot work?"; C "How does openpilot recognize the car model it is connected to?"; D "What should I buy to run openpilot in my car?"; E "How do I update openpilot?"; F "Does openpilot work at all speeds?"; G "What is openpilot?"; H "What are the limitations of openpilot Adaptive Cruise Control?"; I "Do I retain my car factory safety features with openpilot installed?". Accessed October 25, 2026.

[3] H. V. Pham, "Course overview." EECS 4314 Advanced Software Engineering lecture slides, York University, slide 35 ("Architecture Terminology"), 2026.

[4] H. V. Pham, "Software architecture styles." EECS 4314 Lecture 3 slides, York University, 2026.

[5] D. Garlan and M. Shaw, "An introduction to software architecture," in *Advances in Software Engineering and Knowledge Engineering*, vol. 1, World Scientific Publishing, 1993.

[6] H. V. Pham, "Reflexion models." EECS 4314 Advanced Software Engineering lecture slides, York University, slides 3–5, 2026.

[7] Group H, "Shared subsystem names." Internal project naming convention, EECS 4314, Oct. 2026.

[8] A. E. Hassan and R. C. Holt, "A reference architecture for web servers," in *Proceedings of the Seventh Working Conference on Reverse Engineering (WCRE)*, 2000.

[9] H. V. Pham, "Design patterns review." EECS 4314 Advanced Software Engineering lecture slides, York University, 2026.

[10] comma.ai, "What is openpilot?." openpilot docs, https://docs.comma.ai/.

[11] comma.ai, "openpilot glossary (panda, comma connect)." openpilot docs, https://docs.comma.ai/concepts/glossary/.

[12] comma.ai, "Integration with stock features." openpilot docs, https://docs.comma.ai/INTEGRATION/.

[13] comma.ai, "Safety." openpilot docs, https://docs.comma.ai/concepts/safety/.

[14] comma.ai, "Limitations (ALC/LDW, ACC/FCW, DM)." openpilot docs, https://docs.comma.ai/LIMITATIONS/.

[15] comma.ai, "Logging." openpilot docs, https://docs.comma.ai/concepts/logs/.

[16] comma.ai, "What is a car port?." openpilot docs, https://docs.comma.ai/how-to/car-port/.